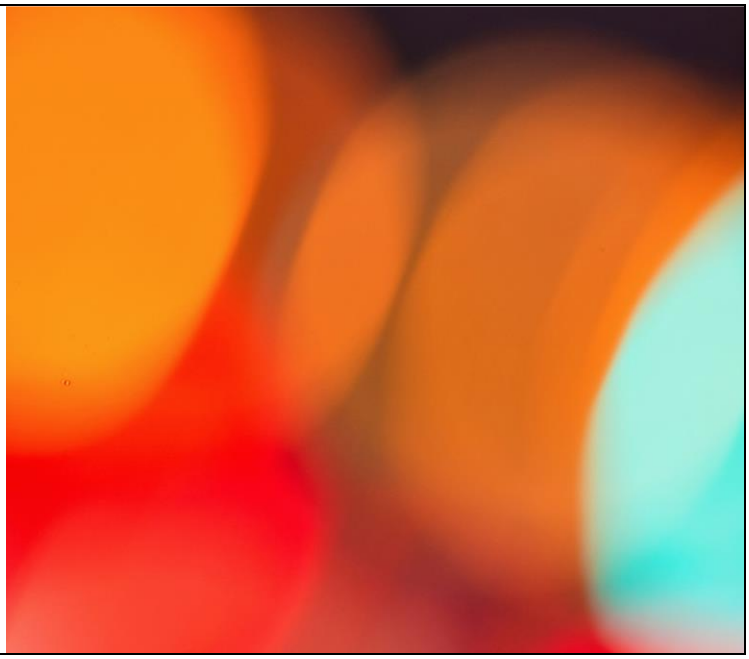
Using Autoencoders for Anomaly Detection

Design and Analysis



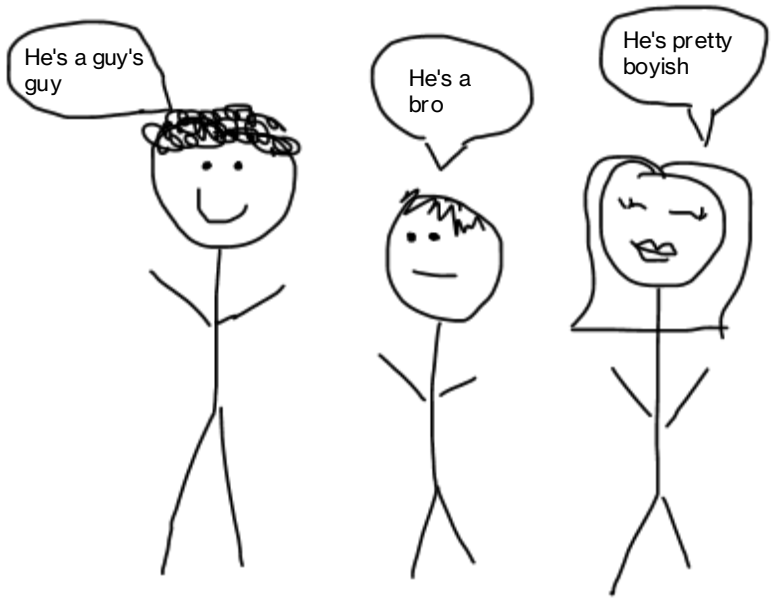
What is an Autoencoder?

To understand our project, we first have to ask: What is an autoencoder, and what is a latent space? Let me introduce you to John. Imagine trying to describe our friend John to someone. John is complex. He has a lot of distinct traits.

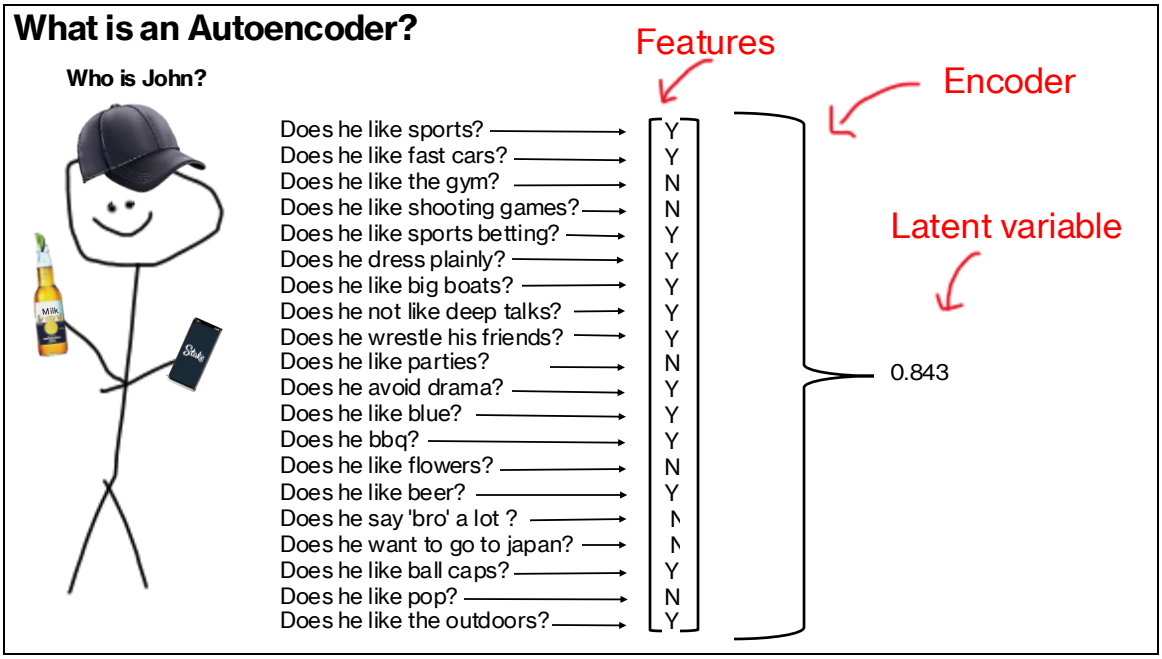
What is an Autoencoder?

To understand our project, we first have to ask: What is an autoencoder, and what is a latent space? Let me introduce you to John. Imagine trying to describe our friend John to someone. John is complex. He has a lot of distinct traits.

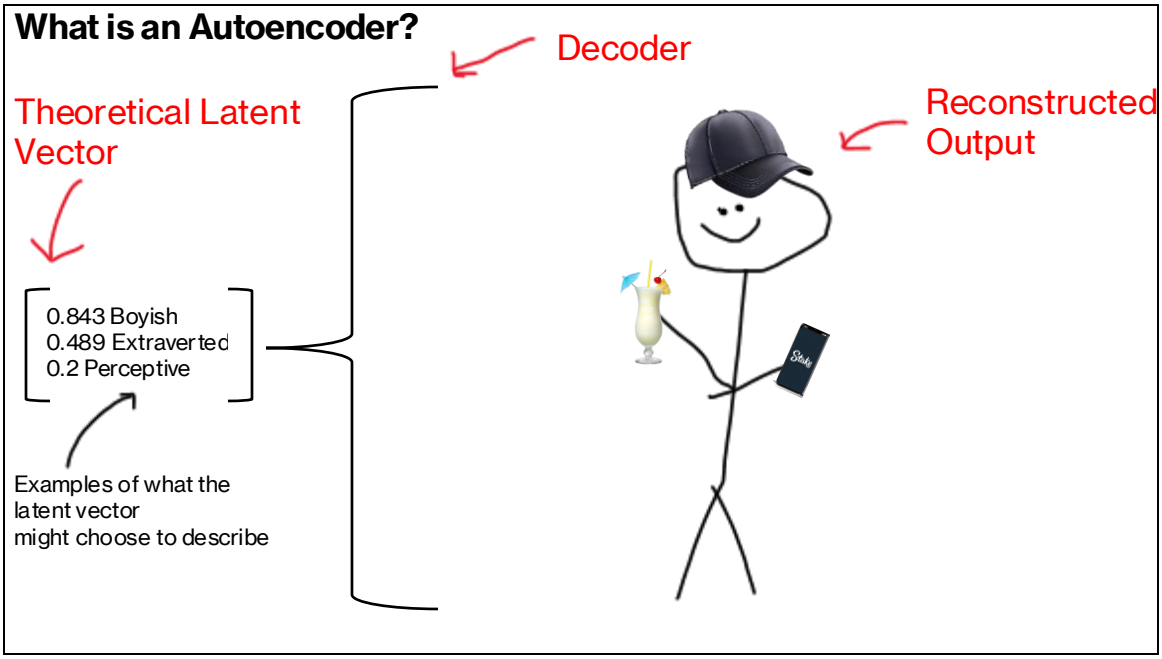
What is an Autoencoder?



Humans naturally compress complex information. If you ask three different people to describe him, they might synthesize all his traits into a single, subjective metric—like a how much of a 'bro' he is.



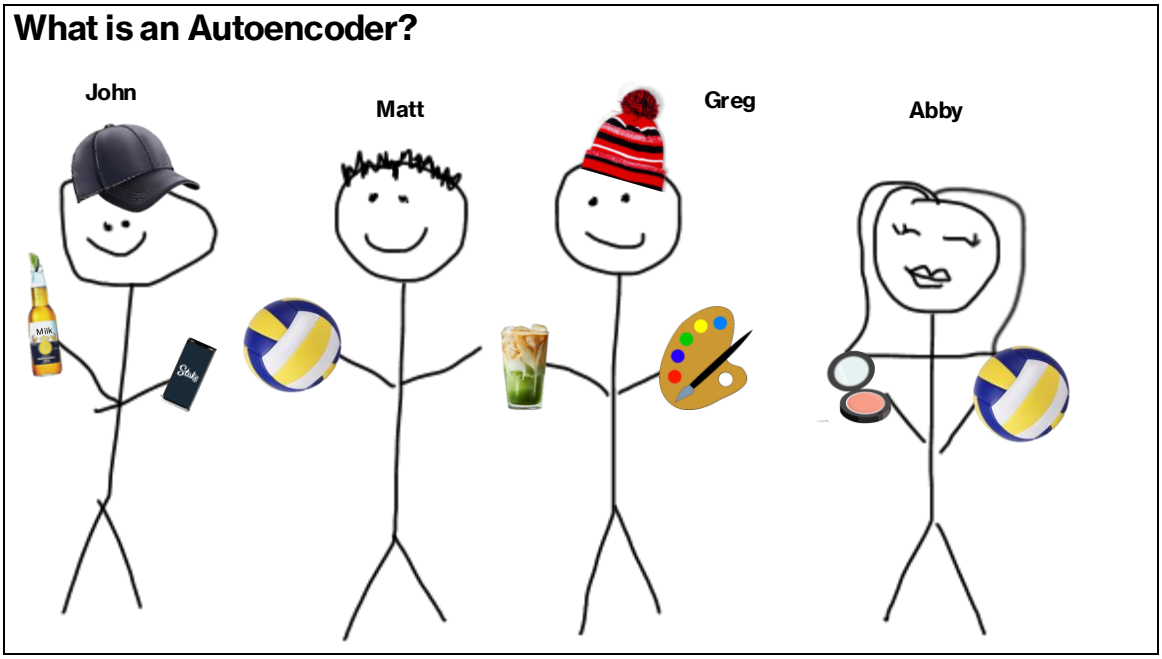
An autoencoder does the exact same thing mathematically. The **Encoder** takes high-dimensional input data, like this questionnaire, and compresses it down through a bottleneck into a **Latent Variable**. Here, our network has compressed all of John's nuances into a single number: 0.843. Which describes something over encompassing of the input data. This is a latent variable in our latent vector.



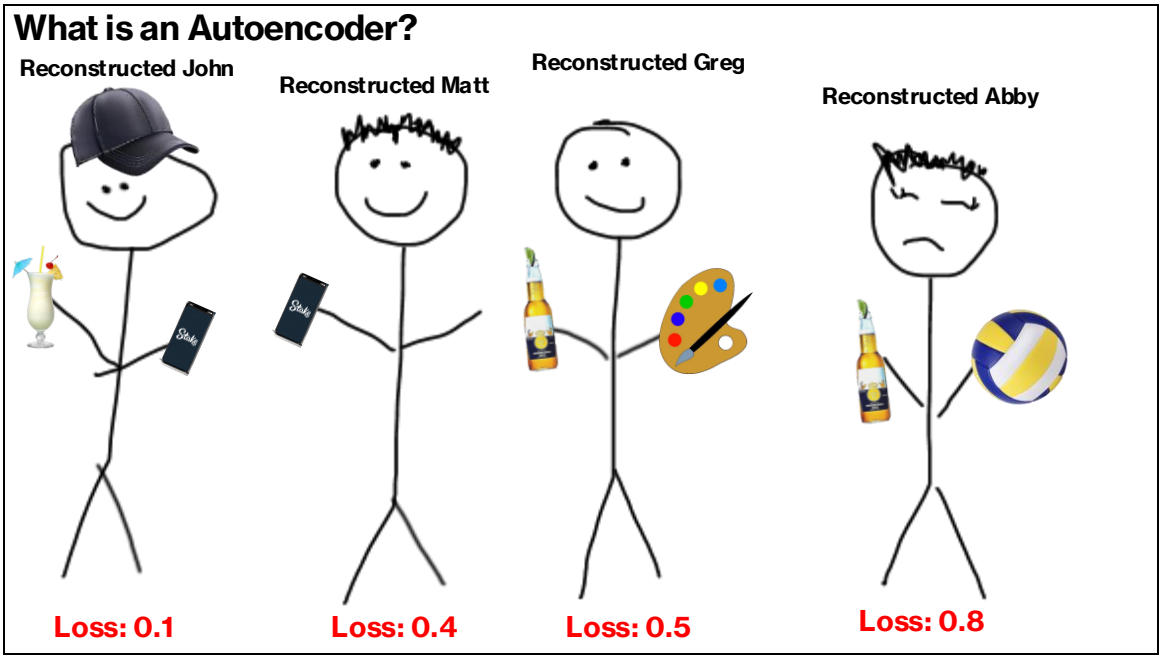
From our latent vector, we hope our decoder is able to reconstruct John.

What is an Autoencoder?

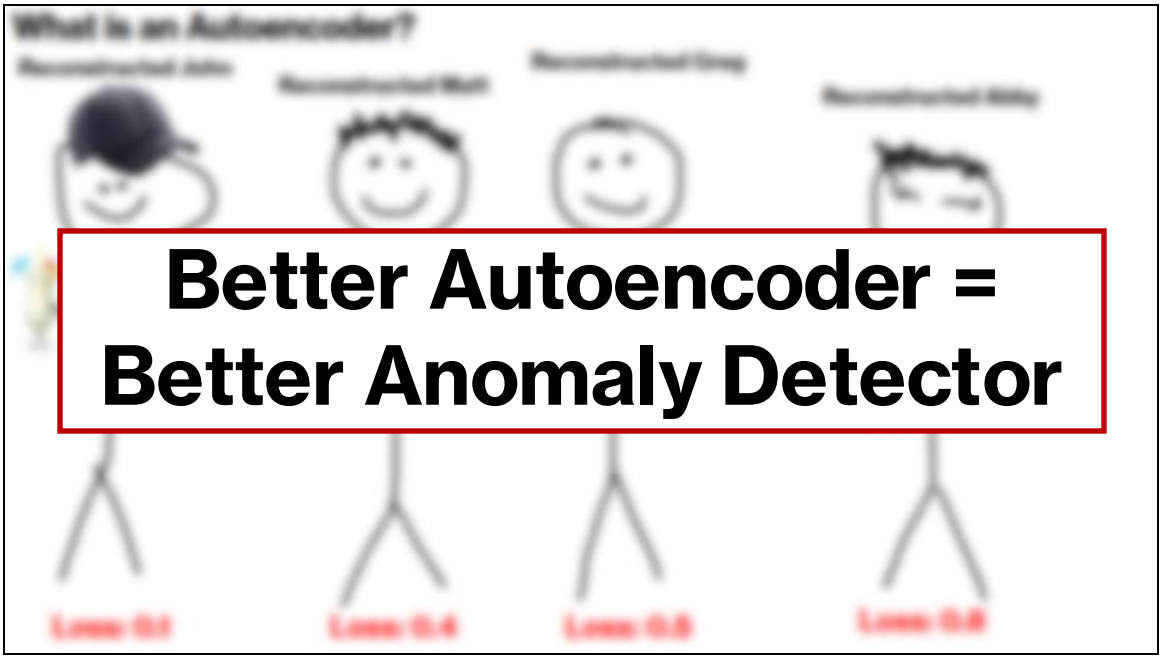
We then measure how well our autoencoder did by a 'reconstruction' loss function. In this example we are using Mean Squared Error



If I give my autoencoder this set of people and ask it to reconstruct them. Given that my auto encoder has been given mainly men, it is very good at reconstructing the men, but is bad at reconstructing the women.



The autoencoders inability to effectively reconstruct the input indicates that this input is unlike the others, and therefore is an anomaly. This is how an autoencoder can be used for anomaly detection.



An autoencoder that is better at reconstructing the normal images will be worse at reconstructing the abnormal images, in turn making anomaly detection better.

Making a better Autoencoder?

So how do we make a better autoencoder?

a disclaimer

While testing the different designs of our autoencoder, we used a starting seed for our weights to get consistent comparable results. Before, some seeds showed way better or way worse output for each of the designs. This shows how much of a difference weight initzialiaton makes. Nevertheless we did not experiment much with our initizialiation.

Making a better Autoencoder: **Convolutional network**

First design	Second design [1]	Third design [2]
Encoder: nn.Conv2d(3,16,3,stride=2,padding=1), nn.ReLU(), nn.Conv2d(16,32,3,stride=2,padding=1), nn.ReLU(), nn.Conv2d(32,64,3,stride=2,padding=1), nn.ReLU(), nn.Conv2d(64,128,3,stride=2,padding=1) nn.ReLU() Decoder reverses encoder	Encoder: nn.Conv2d(3, 32, 3, stride=2, padding=1), nn.LeakyReLU(0.2), nn.BatchNorm2d(32), nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.LeakyReLU(0.2), nn.BatchNorm2d(64), nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.LeakyReLU(0.2), nn.BatchNorm2d(128), nn.Conv2d(128, 256, 3, stride=2, padding=1), nn.LeakyReLU(0.2), nn.BatchNorm2d(256), Decoder reverses encoder	Encoder: (nn.LeakyReLU(0.2) between each layer) nn.Conv2d(3, 32, 4, stride=2, padding=1), nn.Conv2d(32, 32, 4, stride=2, padding=1), nn.Conv2d(32, 32, 3, stride=1, padding=1), nn.Conv2d(32, 64, 4, stride=2, padding=1), nn.Conv2d(64, 64, 3, stride=1, padding=1), nn.Conv2d(64, 128, 4, stride=2, padding=1), nn.Conv2d(128, 64, 3, stride=1, padding=1), nn.Conv2d(64, 32, 3, stride=1, padding=1), nn.Conv2d(32, latent_vector_dim, 14, stride=1, padding=0), Decoder reverses encoder

We first started with experimenting with the convolutional network

Making a better Autoencoder: **Convolutional network**

AUROC SCORES

First Design
AUROC Scores

Screws	Pasta	Capsule
0.48	0.55	0.45

Second Design
AUROC Scores

Screws	Pasta	Capsule
0.52	0.67	0.67

Third Design
AUROC Scores

Screws	Pasta	Capsule
0.72	0.43	0.66

For all using Mean Squared Error and 30 epochs

This is what we found for this experimentation

Tighter bottle neck, leakyReLU, Batch norm
= better results

We first started with experimenting with the convolutional network

Different Autoencoder designs: **MSE vs SSIM loss**

Using second convolutional design as benchmark

MSE loss equation

$$\mathrm{MSE} = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

Basic Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.52	0.67	0.67

SSIM loss equation

$$\mathrm{SSIM}(\mathbf{x},\mathbf{y}) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)\,(\sigma_x^2 + \sigma_y^2 + C_2)}.$$

Basic Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.64	0.78	0.53

We then moved on to... SSIM over MSE is that it (Using an SSIM loss functionresults in a more meaningfullatent space for images)blah blah while MSE blah blah. When we tested the results we found

Different Autoencoder designs: **Changing hyperparameters**

Using 2nd design as benchmark

MSE, 30 epochs, 1e-1 learning rate

Screws	Pasta	Capsule
0.68	0.45	0.31

MSE, 30 epochs, 1e-2 learning rate

Screws	Pasta	Capsule
0.52	0.67	0.67

MSE, 30 epochs, 1e-3 learning rate

Screws	Pasta	Capsule
0.52	0.5	0.68

MSE, 50 epochs, 1e-2 learning rate

Screws	Pasta	Capsule
0.64	0.48	0.56

MSE, 50 epochs, 1e-3 learning rate

Screws	Pasta	Capsule
0.42	0.25	0.62

SSIM, 30 epochs, 1e-2 learning rate

Screws	Pasta	Capsule
0.64	0.78	0.53

SSIM, 30 epochs, 1e-3 learning rate

Screws	Pasta	Capsule
0.66	0.5	0.56

We then moved on to... SSIM over MSE is that it (Using an SSIM loss functionresults in a more meaningfullatent space for images)blah blah while MSE blah blah. When we tested the results we found

Different Autoencoder designs: **Changing hyperparameters**

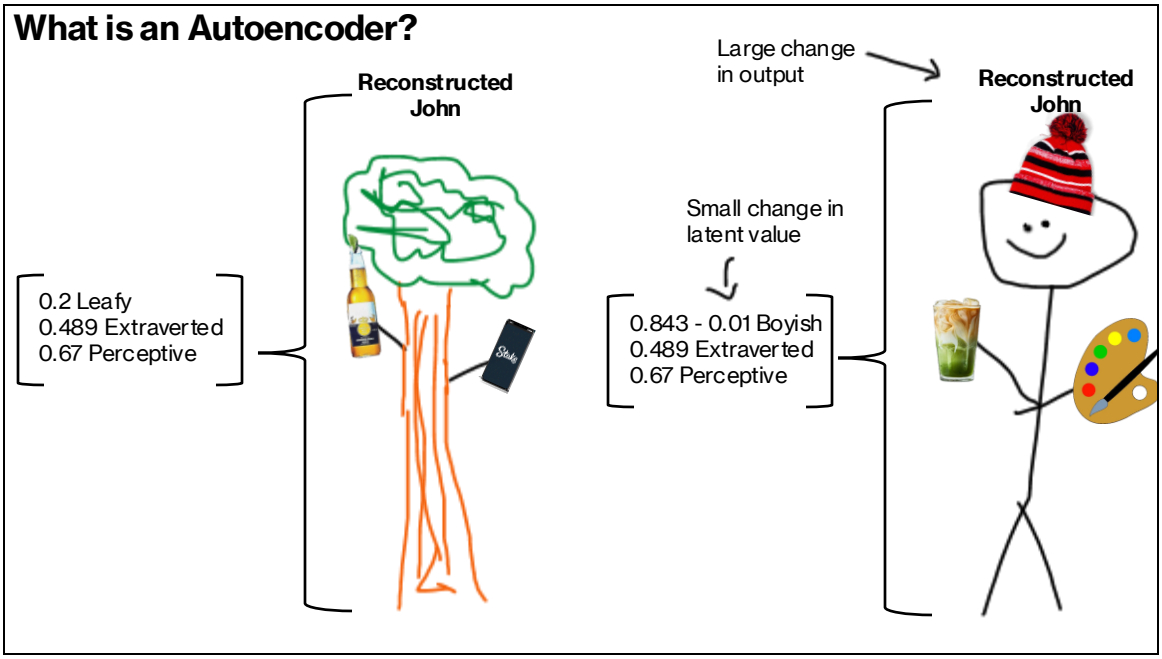
Using 2nd design as benchmark

- Too many epochs -> lower scores because overfitting (around 30 epochs was found to be best
- Learning rate around 1e-2 generally best, most consistent (failing to converge with higher learning rate -> loss grows exponentially particularly for 3rd design
- SSIM better particularly for pasta
- MSE better for capsules
- Next step would be to try hybrid loss function

We then moved on to... SSIM over MSE is that it (Using an SSIM loss functionresults in a more meaningfullatent space for images)blah blah while MSE blah blah. When we tested the results we found

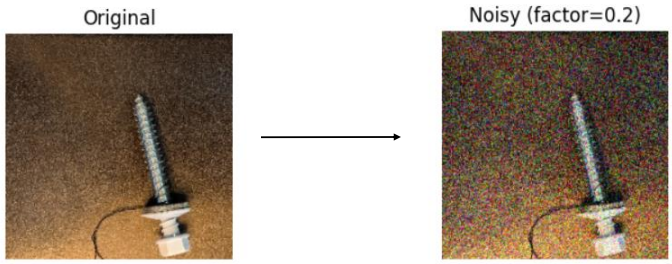
Different Autoencoder Designs

We will now present a couple of designs we looked at to improve the performance of the Autoencoder



Before I continue. What these different designs I am about to present to you attempt to improve upon is the latent space I talked about. If our latent vector parameters are ineffective at describing the input, or the input space does not vary in a logical way, then we will have an ineffective autoencoder.

Different Autoencoder designs: **Denoising Autoencoder**



SSIM Loss, learning rate: 1e-3, latent vector dim: 32, epochs: 35, noise factor: 0.02

Basic Autoencoder
AUROC Scores

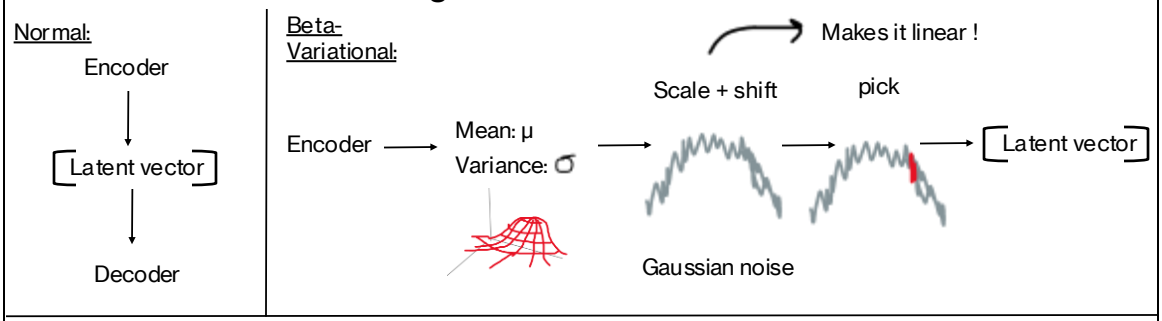
Screws	Pasta	Capsule
0.66	0.5	0.56

Denoising Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.7	0.53	0.61

Another variation is a denoising autoencoder. This adds gaussian noise to the input image and then calculates the loss by comparing the output to the original input image. Because you compare the non-noisy input to the output in your loss function, this forces the model to learn more about the underlying nature of the input to reduce the loss function, which reduces overfitting. **Comment on results**

Different Autoencoder designs: **Beta Variational Autoencoder**



Loss = SSIM + β * (KL Divergence)

The variational autoencoder is a design where instead of outputting a latent variable, the encoder outputs a mean and a variance. This yields a more robust latent space as the model is forced to learn an entire neighborhood of valid representations, meaning it won't suddenly fail if it encounters data that slightly deviates from the training set. The KL divergence **acts as a regularization penalty**, penalizing the model for a distribution that does not well fit a standard distribtuion. We scale it beta as we still want the model to prioritize the reconstruction loss.

Different Autoencoder designs: **Beta Variational Autoencoder**

SSIM Loss, ~~learning rate: 1e-3~~, latent vector dim: 32, epochs: 35, noise factor: 0.02, beta: 0.01

Learning rate for B-VAE: 0.5e-4

Basic Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.66	0.5	0.56

Denoising Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.7	0.53	0.61

Beta-Variational Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.68	0.65	0.44

Different Autoencoder designs: **Beta Variational Denoising Autoencoder**

SSIM Loss, learning rate: 1e-3 / 0.5e-4, latent vector dim: 32, epochs: 35, noise factor: 0.02, beta: 0.01

Denoising Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.7	0.53	0.61

+

Beta-Variational Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.68	0.65	0.44

=

Beta-Variational Autoencoder
AUROC Scores

Screws	Pasta	Capsule
0.7	0.3 !	0.48

Our best designs so far were the denoising and beta-variational autoencoder. So surely combining these will make the best autoencoder? Not quite. While combining them sounds good in theory, these two architectures actually fight each other. Denoising needs to hold onto complex features to rebuild the corrupted image, but the Beta-VAE is strictly trying to compress and smooth out the latent space. The resulting model is totally over-regularized—it just blurs away the exact fine textures we need to spot defects in things like pasta

Our final design?

To wrap up: there is no universal 'best' model. It all depends on your data.
For example, simple screws worked fine with standard MSE loss. But the pasta has complex textures which SSIM did a better job picking up on. Beyond that, the design space is massive. Fully optimizing all the parameters and architectures and weight initialization would take significantly more time.
Finally, our biggest bottleneck is data size. Our immediate next step would be getting larger datasets to really improve our scores.

References

[1] V. Thakkar, S. Tewary and C. Chakraborty, "Batch Normalization in Convolutional Neural Networks – A comparative study with CIFAR-10 data," 2018 Fifth International Conference on Emerging Applications of Information Technology (EAIT), Kolkata, India, 2018, pp. 1-5, doi: 10.1109/EAIT.2018.8470438. keywords: {Training;Machine learning;Convolutional neural networks;Data models;Atmospheric modeling;Task analysis;Analytical models;Activation function;Batch normalization;Convolutional Neural Networks;Deep Learning;and Image Analysis.},

[2] Bergmann, P., Löwe, S., Fauser, M., Sattlegger, D., & Steger, C. (2018). Improving Unsupervised Defect Segmentation by Applying Structural Similarity to Autoencoders. *ArXiv*. https://doi.org/10.5220/0007364503720380